

Assignment Completion Sheet

USCSP 359			
Practical course in Object Oriented Programming using Java I			
Assignment No	Assignment Name	Marks (out of 5)	Teacher's Sign
1	Java tools, Basics of Java and Classes and Objects		
2	Array of Objects and Packages		
3	Inheritance and Interface		
4	Exception handling and File Handling		
5	GUI designing and Event handling		
Total (Out of 25)			
Lab book (Out of 10)			
Viva/Quiz (Out of 5)			
Internal (Out of 15)			

CERTIFICATE

This is to certify that Mr./Ms. _____ has
successfully completed **TYBSc CS USCSP 359 Practical course in**
Object Oriented Programming using Java I Semester V in the
year _____ and his/her seat number is _____.
He / She have scored _____ marks out of 15.

Instructor

HOD/Coordinator

Internal Examiner

External Examiner

Assignment 1: Java Tools, Basics of Java and Classes and Objects

Aim: The aim of learning Java tools, Basics of Java, Classes and Objects is to understand and effectively utilize the software and utilities that facilitate Java development. This includes integrated development environments (IDEs), build automation tools, version control systems, and debugging utilities.

Theory: Java is extensively used programming language. It is been used by many programmers from beginners to experts. If we look into the phases of program development, we come across many phases such as:

Step 1: Creation of a Java program: Creating a Java program means writing of a Java program on any editor or IDE. After creating a Java program provide .java extension to file. It signifies that the particular file is a Java source code. Also, whatever progress we do from writing, editing, storing and providing .java extension to a file, it basically comes under creating of a Java program.

Step 2: Compiling a Java Program: Our next step after creation of program is the compilation of Java program. Generally Java program which we have created in step 1 with a .java extension, it is been compiled by the compiler. Suppose if we take example of a program say Welcome.java, when we want to compile this we use command such as javac. After opening command prompt or shell console we compile Java

program with .java extension as: "javac Welcome.java"

After executing the javac command, if no errors occur in our program we get a .class file which contains the bytecode. It means that the compiler has successfully converted Java source code to bytecode. Generally bytecode is been used by the JVM to run a particular application. The final bytecode created is the executable file which only JVM understands. As Java compiler is invoked by javac command, the JVM is been invoked by java command. On the console window you have to type: "java Welcome"

This command will invoke the JVM to take further steps for execution of Java program.

Step 3: Program loading into memory by JVM: JVM requires memory to load the .class file before its execution. The process of placing a program in memory in order to run is called as Loading. There is a class loader present in the JVM whose main functionality is to load the .class file into the primary memory for the execution. All the .class files required by our program for the execution is been loaded by the class loader into the memory just before the execution.

Step 4: Bytecode Verification by JVM: In order to maintain security of the program JVM has bytecode verifier. After the classes are loaded in to memory, bytecode verifier comes into picture and verifies bytecode of the loaded class in

order to maintain security. It check whether bytecodes are valid. Thus it prevent our computer from malicious viruses and worms.

Step 5: Execution of Java program: Whatever actions we have written in our Java program, JVM executes them by interpreting bytecode. JVM uses JIT compilation unit to which we even call just-in-time compilation.

JVM (Java Virtual Machine): JVM, i.e., Java Virtual Machine. JVM is the engine that drives the Java code. Mostly in other Programming Languages, compiler produce code for a particular system but Java compiler produce Bytecode for a Java Virtual Machine. When we compile a Java program, then bytecode is generated. Bytecode is the source code that can be used to run on any platform. Bytecode is an intermediary language between Java source and the host system. It is the medium which compiles Java code to bytecode which gets interpreted on a different machine and hence it makes it Platform/Operating system independent.

A Simple Java Program:

```
public class Test
{
    public static void main(String args[])
    {
        System.out.println("Hello Students");
    }
}
```

class keyword is used to declare a class in Java.

public keyword is an access modifier that represents visibility. It means it is visible to all.

static is a keyword. If we declare any method as static, it is known as the static method. The core advantage of the static method is that there is no need to create an object to invoke the static method. The main () method is executed by the JVM, so it doesn't require creating an object to invoke the main () method. So, it saves memory.

void is the return type of the method. It means it doesn't return any value.

main represents the starting point of the program.

String[] args or String args[] is used for command line argument.

System.out.println() is used to print statement. Here, System is a class, out is an object of the PrintStream class, println() is a method of the PrintStream class.

To compile the program the command is: javac Test.java

To execute the program the command is: java Test

Different ways of reading input from console in Java -

1. Using command line argument: The java command-line argument is an argument i.e. passed at the time of running the java program. The arguments

passed from the console can be received in the java program and it can be used as an input.

```
public class Test
{
    public static void main(String args[])
    {
        int a,b,c;
        a = Integer.parseInt(args[0]);
        b = Integer.parseInt(args[1]);
        c = a + b;
        System.out.println("Addition: "+c);
    }
}
```

2. Using Buffered Reader Class: This is the Java classical method to take input. This method is used by wrapping the System.in (standard input stream) in an InputStreamReader which is wrapped in a BufferedReader, we can read input from the user in the command line.

```
public class Test
{
    public static void main(String args[])
    {
        int rollno;
        String name;
        BufferedReader br = new BufferedReader(new
InputStreamReader(System.in));
        System.out.println("Enter roll number:");
        rollno = Integer.parseInt(br.readLine());
        System.out.println("Enter name:");
        name = br.readLine();
        System.out.println(" Roll Number = " + rollNumber);
        System.out.println(" Name = " + name);
    }
}
```

3. Using Scanner Class: This is probably the most preferred method to take input. The main purpose of the Scanner class is to parse primitive types and strings using regular expressions, however it also can be used to read input from the user in the command line.

```
import java.util.Scanner;
public class ScannerTest
{
```

```

public static void main(String args[])
{
    Scanner sc=new Scanner(System.in);
    System.out.println("Enter your rollno and name :");
    int rollno=sc.nextInt();
    String name=sc.next();
    System.out.println("Rollno:"+rollno+" Name:"+name);
    sc.close();
}
}

```

Class: Classes are the fundamental building blocks of any object-oriented language. When you define a class, you declare its exact form and nature. You do this by specifying the data that it contains and the code that operates on that data. While very simple classes may contain only code or only data, most real-world classes contain both. A class is declared by use of the class keyword. A simplified general form of a class definition is shown here:

```

accessspecifier class classname
{
    DataType instancevariable1;
    DataType instancevariable 2;
    DataType instancevariable n;
    returntype methodname1 (parameter list)
    {
        Body of the method;
    }
    returntype methodname2 (parameter list)
    {
        Body of the method;
    }
}

```

The data, or variables, defined within a class are called instance variables. The code is contained within methods. Collectively, the methods and variables defined within a class are called members of the class. Variables defined within a class are called instance variables because each instance of the class contains its own copy of these variables. Thus, the data for one object is separate and unique from the data for another.

Objects: When you create a class, you are creating a new data type. You can use this type to declare objects of that type. However, obtaining objects of a class is a two-step process.

First, you must declare a variable of the class type. This variable does not define an

object. Instead, it is simply a variable that can refer to an object. Second, you must acquire an actual, physical copy of the object and assign it to that variable. You can do this using the new operator. The new operator dynamically allocates memory for an object and returns a reference to it.

This reference is, essentially, the address in memory of the object allocated by new. This reference is then stored in the variable. Thus, in Java, all class objects must be dynamically allocated.

Syntax for declaring Object: `Student stud= new Student();`

This statement combines the two steps just described.

It can be rewritten like this to show each step more clearly:

`Student stud;// declare reference to object`

`Stud = new Student (); // allocate a object`

The first line declares stud as a reference to an object of type Student. At this point, stud does not yet refer to an actual object. The next line allocates an object and assigns a reference to it to stud. After the second line executes, you can use stud as if it were a Student object. But in reality, stud simply holds, in essence, the memory address of the actual Student object.

Array of Objects: An array that contains class type elements are known as an array of objects. It stores the reference variable of the object. Before creating an array of objects, we must create an instance of the class by using the new keyword. Suppose, we have created a class named Student. We want to keep marks of 20 students. In this case, we will not create 20 separate objects of Student class.

Instead of this, we will create an array of objects, as follows:

`Student s[] = new Student[20];`

```
public class Student
{
```

```
    int marks;
```

```
}
```

```
public class ArrayOfObjects
```

```
{
```

```
    public static void main(String args[])
```

```
    {
```

```
        Student std[] = new Student[3]; // array of reference variables of
Student
```

```
        std[0] = new Student(); // convert each reference variable into Student object
```

```
        std[1] = new Student();
```

```
        std[2] = new Student();
```

```
        std[0].marks = 40; // assign marks to each Student element
```

```
        std[1].marks = 50;
```

```
        std[2].marks = 60;
```

```

        System.out.println("\n students average marks:"
+(std[0].marks+std[1].marks+std[2].marks)/3);
    }
}

```

SET A

1. Check the methods of String, Scanner, Integer class using javap command.
2. Write a Java program to accept a number from the user and display a multiplication table of a number. Accept number using Scanner class.
3. Write a Java Program to Reverse a Number. Accept number using command line argument.
4. Write a Java program to print the sum of elements of the array. Accept 'n' array elements from the user.
5. Write a menu driven program to perform the following operations.
 - i. Calculate the volume of the cylinder. (hint : Volume: $\pi \times r^2 \times h$)
 - ii. Find the factorial of a given number.
 - iii. Check if the number is Armstrong or not.
 - iv. Exit
6. Write a program to accept 'n' names of countries and display them in descending order.
7. Write a Java program to display prime numbers in the range from 1 to 100.

SET B

1. Write a Java program create class as MyDate with dd,mm,yy as data members. Write accept and display methods. Display the date in dd-mm-yy format.
2. Write a java program which define class Employee with data member as name and salary. Program store the information of 5 Employees. (Use array of object)
3. Write a java program to create class Account (accno, accname, balance). Create an array of "n" Account objects. Define the static method "sortAccount" which sorts the array on the basis of balance. Display account details in sorted order.

SET C

1. Define a class Person (id, name, dateofbirth). For dateofbirth create an object of MyDate class which is created in SET B 1. Define default and parameterized constructor. Also define accept and display function in Person and MyDate class. Call constructor and function of MyDate class from Person class for dateofbirth. Accept the details of n person and display the

details.

Sample code:

```
public class Person
{
    private int id;
    private String name;
    private MyDate dob;
    private static int cnt=1;
    -----
    -----
}
```

Signature of instructor: _____ Date: _____

Assignment Evaluation

0: Not Done []

1: Incomplete []

2: Late Complete []

3: Needs Improvement []

4: Complete []

5: Well done []

Assignment 2: Array of Objects and Packages

Aim: Constructors set the initial state of an object and can ensure that the object starts in a consistent and valid state. Packages in Java are used to group related classes, interfaces, and sub-packages. The aim of using packages is to organize code, avoid naming conflicts, and control access.

Theory:

Array of Objects: An array that contains class type elements are known as an array of objects. It stores the reference variable of the object. Before creating an array of objects, we must create an instance of the class by using the new keyword. Suppose, we have created a class named Student. We want to keep marks of 20 students. In this case, we will not create 20 separate objects of Student class. Instead of this, we will create an array of objects, as follows:

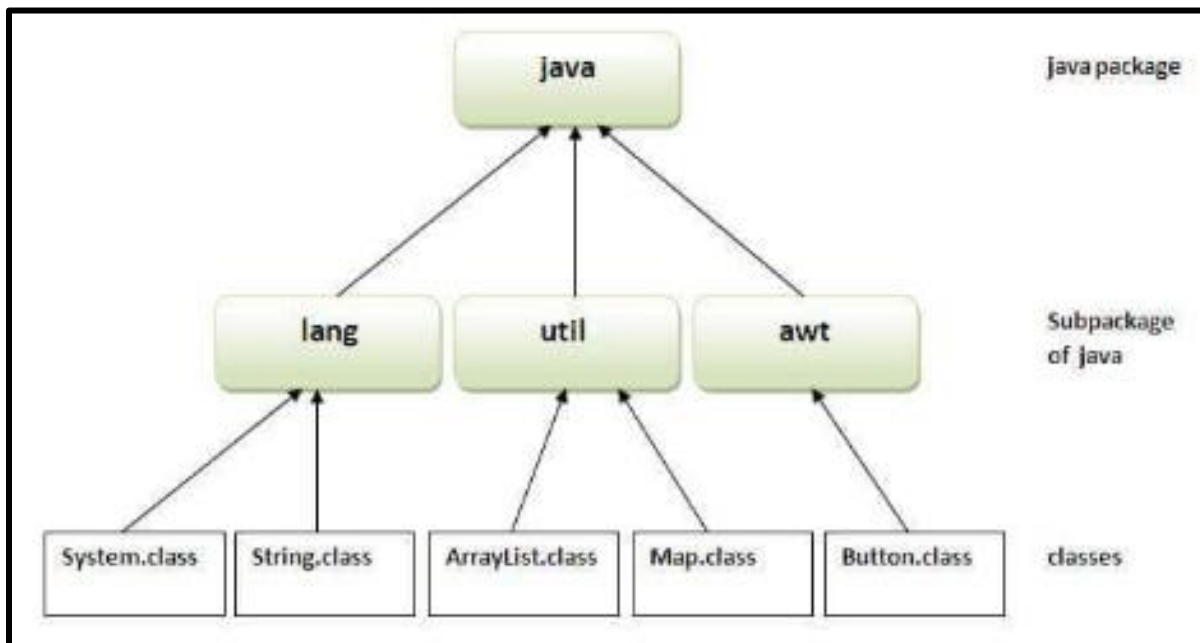
```
Student s[] = new Student[20];
public class Student
{
    int marks;
}
public class ArrayOfObjects
{
    public static void main(String args[])
    {
        Student std[] = new Student[3]; // array of reference variables of
Student
        std[0] = new Student(); // convert each reference variable into Student object
        std[1] = new Student();
        std[2] = new Student();
        std[0].marks = 40; // assign marks to each Student element
        std[1].marks = 50;
        std[2].marks = 60;
        System.out.println("\n students average marks:"
+(std[0].marks+std[1].marks+std[2].marks)/3);
    }
}
```

Package: A java package is a group of similar types of classes, interfaces and sub-packages. Package in java can be categorized in two form, built-in package and user-defined package. There are many built-in packages such as java, lang, awt,

javax, swing, net, io, util, sql etc.

Advantage of Java Package:

- 1) Java package is used to categorize the classes and interfaces so that they can be easily maintained.
- 2) Java package provides access protection.
- 3) Java package removes naming collision.



Simple example of java package: The package keyword is used to create a package in java.

```
package mypack;
public class Simple
{
    public static void main(String args[])
    {
        System.out.println("Welcome to package");
    }
}
```

How to compile java package: If you are not using any IDE, you need to follow the syntax given below: `javac -d directory javafilename`

For example: `javac -d . Simple.java`

The `-d` switch specifies the destination where to put the generated class file. You can use any directory name like `/home` (in case of Linux), `d:/abc` (in case of windows) etc. If you want to keep the package within the same directory, you can use `.` (dot).

How to run java package program - You need to use fully qualified name e.g. mypack.Simple etc to run the class.

SET A

1. Write a java program to take 5 integres as input and store it in a array also print the array.
2. Write a java program to reverse the array.
3. Create a package named “Series” having three different classes to print series:
 - i. Fibonacci series
 - ii. Cube of numbers
 - iii. Square of numbers
4. Write a java program to generate “n” terms of the above series. Accept n from user.

SET B

1. Create a package “utility”. Define a class CapitalString under “utility” package which will contain a method to return String with first letter capital. Create a Person class (members– name, city) outside the package. Display the person name with first letter as capital by making use of CapitalString.
2. Write a package for String operation which has two classes Con and Comp. Con class has to concatenate two strings and comp class compares two strings. Also display proper message on execution.

SET C

1. Write a Java program to create a Package “SY” which has a class SYMarks (members – ComputerTotal, MathsTotal, and ElectronicsTotal). Create another package TY which has a class TYMarks (members – Theory, Practicals). Create n objects of Student class (having rollNumber, name, SYMarks and TYMarks). Add the marks of SY and TY computer subjects and calculate the Grade (‘A’ for ≥ 70 , ‘B’ for ≥ 60 ‘C’ for ≥ 50 , Pass Class for ≥ 40 else ‘FAIL’) and display the result of the student in proper format.
2. Write a java program to search the given number in array and output position of given number.
3. Write a java to search given number in array use binary search.

Signature of instructor: _____ Date: _____

Assignment Evaluation

0: Not Done []

1: Incomplete []

2: Late Complete []

3: Needs Improvement []

4: Complete []

5: Well done []

Assignment 3: Inheritance and Interface

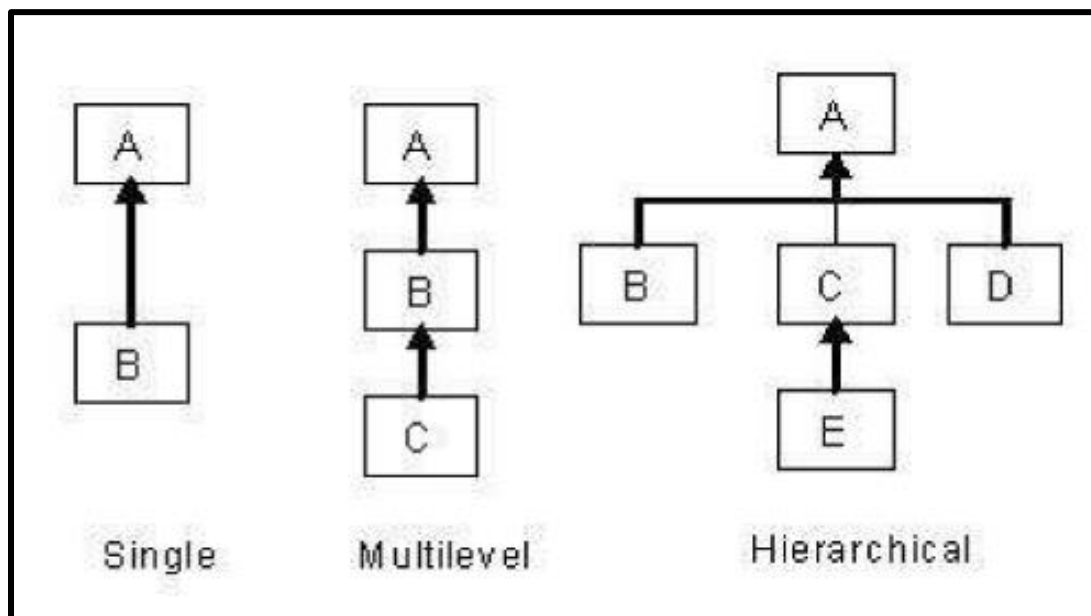
Aim: Inheritance allows a new class (subclass) to acquire the properties and behaviors of an existing class (superclass), enabling developers to create more complex systems and extend existing functionality without modifying the original class. Interfaces define a contract for what a class can do, without dictating how it should do it.

Theory:

Inheritance: Inheritance is one of the feature of Object Oriented Programming System (OOPs) it allows the child class to acquire the properties (the data members) and functionality (the member functions) of parent class.

What is child class?: A class that inherits another class is known as child class, it is also known as derived class or subclass.

What is parent class?: The class that is being inherited by other class is known as parent class, super class or base class.



Access in subclass: The following members can be accessed in a subclass:

- i) public or protected superclass members.
- ii) Members with no specifier if subclass is in same package.

The use of super keyword:

- i) Invoking superclass constructor - `super(arguments)`
- ii) Accessing superclass members – `super.member`
- iii) Invoking superclass methods – `super.method(arguments)`

Example:

```

public class A
{
    protected int num;
    public A(int num)
    {
        this.num = num;
    }
}
public class B extends A
{
    private int num;
    public B(int a, int b)
    {
        super(a); //should be the first line in the subclass constructor
        this.num = b;
    }
    public void display()
    {
        System.out.println("In A, num = " + super.num);
        System.out.println("In B, num = " + num);
    }
}

```

Overriding methods: Redefining superclass methods in a subclass is called overriding. The signature of the subclass method should be the same as the superclass method.

Example:

```

public class A
{
    public void display(int num) { //code }
}
public class B extends A
{
    public void display(int x) { //code }
}

```

Dynamic binding: When over-riding is used, the method call is resolved during run-time i.e. depending on the object type, the corresponding method will be invoked.

Example:

```

A ref;
ref = new A();

```

```
ref.display(10); //calls method of class A
```

```
ref = new B();
```

```
ref.display(20); //calls method of class B
```

Final Keyword in Java: The final keyword in java is used to restrict the user. The java final keyword can be used in many context. final keyword can be for:

variable

method

class

The final keyword can be applied with the variables, a final variable that have no value it is called blank final variable or uninitialized final variable. It can be initialized in the constructor only. The blank final variable can be static also which will be initialized in the static block only. We will have detailed learning of these. Let's first learn the basics of final keyword.

1) Java final variable: If any variable as final, value of final variable cannot change the (It will be constant).

2) Java final method: If any method as final, that method cannot be overridden in subclass.

3) Java final class: If any class as final, then that class cannot be extended ie creating subclass of final class is not allowed.

Abstract class: An abstract class is a class which cannot be instantiated. It is only used to create subclasses. A class which has abstract methods must be declared abstract. An abstract class can have data members, constructors, method definitions and method declarations.

```
abstract accessspecifier class ClassName
```

```
{
```

```
...
```

```
}
```

Abstract method: An abstract method is a method which has no definition. The definition is provided by the subclass. abstract accessspecifier returnType method(arguments);

Interface: An interface in Java is a blueprint of a class. It has static constants and abstract methods. The interface in Java is a mechanism to achieve abstraction.

There can be only abstract methods in the Java interface, not method body. It is used to achieve abstraction and multiple inheritance in Java. In other words, you can say that interfaces can have abstract methods and variables. It cannot have a method body.

```
accessspecifier interface InterfaceName
```

```
{
```

```
    //final variables
```

```
    //abstract methods
```

```

}
Example:
public interface MyInterface
{
    int size= 10; //final and static
    void method1();
    void method2();
}
public Class MyClass implements MyInterface
{
    //define method1 and method2
}

```

SET A

1. Define a “Point” class having members – x,y(coordinates). Define default constructor and parameterized constructors. Define two subclasses “ColorPoint” with member as color and subclass “Point3D” with member as z (coordinate). Write display method to display the details of different types of Points.
2. Define a class Employee having members – id, name, salary. Define default constructor. Create a subclass called Manager with private member bonus. Define methods accept and display in both the classes. Create “n” objects of the Manager class and display the details of the worker having the maximum total salary (salary + bonus).
3. Create an abstract class “order” having members id, description. Create two subclasses “Purchase Order” and “Sales Order” having members customer name and Vendor name respectively. Define methods accept and display in all cases. Create 3 objects each of Purchase Order and Sales Order and accept and display details.

SET B

1. Define an interface “Operation” which has methods area(),volume(). Define a constant PI having a value 3.142. Create a class circle (member – radius), cylinder (members – radius, height) which implements this interface. Calculate and display the area and volume.
2. Write a Java program to create a super class Employee(members – name, salary). Derive a sub-class as Developer(member – projectname). Derive a sub-class Programmer(member – proglanguage) from Developer. Create object of Developer and display the details of it. Implement this multilevel inheritance with appropriate constructor and methods.

3. Define an abstract class Staff with members name and address. Define two sub-classes of this class – FullTimeStaff(members - department, salary, hra - 8% of salary, da – 5% of salary) and PartTimeStaff(members - number-of-hours, rate-per-hour). Define appropriate constructors. Write abstract method as calculateSalary() in Staff class. Implement this method in subclasses. Create n objects which could be of either FullTimeStaff or PartTimeStaff class by asking the user's choice. Display details of all FullTimeStaff objects and all PartTimeStaff objects along with their salary.

SET C

1. Create an interface — CreditCardInterface with method: viewCreditAmount(), useCard(), payCredit() and increaseLimit(). Create a class SilverCardCustomer (name, cardnumber (16digits), creditAmount – initialized to 0, creditLimit-set to 50,000) which implements the above interface. Inherit class GoldCardCustomer from SilverCardCustomer having the same methods but creditLimit of 1,00,000. Create an object of each class and perform operations. Display appropriate messages for success or failure of transactions.(Use method overriding)
 - i. useCard() method increases the creditAmount by a specific amount upto creditLimit.
 - ii. payCredit() reduces the creditAmount by a specific amount.
 - iii. increaseLimit() increases the creditLimit for GoldCardCustomers (only 3 times, not more than 5000Rs. each time).

Signature of instructor: _____ Date: _____

Assignment Evaluation

0: Not Done []	1: Incomplete []	2: Late Complete []
3: Needs Improvement []	4: Complete []	5: Well done []

Assignment 4: Exception Handling and File Handling

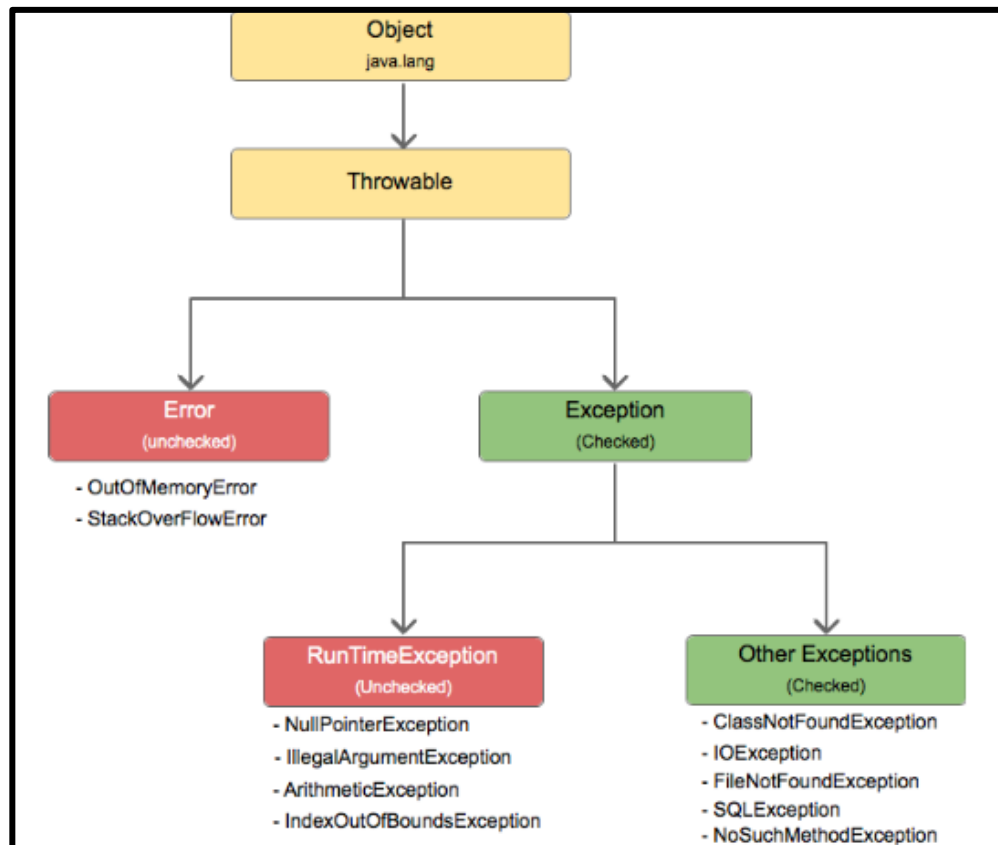
Aim: Exception handling helps in maintaining the normal flow of the application even when unexpected situations occur. File handling is essential for reading, writing, and manipulating files, which is a common requirement in many applications for data storage, configuration, and data processing.

Theory:

Exception Handling: Exception Handling is the mechanism to handle runtime malfunctions. We need to handle such exceptions to prevent abrupt termination of program. The term exception means exceptional condition, it is a problem that may arise during the execution of program. A bunch of things can lead to exceptions, including programmer error, hardware failures, files that need to be opened cannot be found, resource exhaustion etc.

Exception: A Java Exception is an object that describes the exception that occurs in a program. When an exceptional event occurs in java, an exception is said to be thrown. The code that's responsible for doing something about the exception is called an exception handler.

Exception class Hierarchy: All exception types are subclasses of class Throwable, which is at the top of exception class hierarchy.



Exception class is for exceptional conditions that program should catch. This class is extended to create user specific exception classes. RuntimeException is a subclass of Exception. Exceptions under this class are automatically defined for programs. Exceptions of type Error are used by the Java run-time system to indicate errors having to do with the run-time environment, itself. Stack overflow is an example of such an error.

Types of Java Exceptions: There are mainly two types of exceptions: checked and unchecked. An error is considered as the unchecked exception.

1. Checked Exception: The classes that directly inherit the Throwable class except RuntimeException and Error are known as checked exceptions.

Checked exceptions are checked at compile-time.

Checked Exceptions:

- Exception
- IOException
- FileNotFoundException
- ParseException
- ClassNotFoundException
- CloneNotSupportedException
- InstantiationException
- InterruptedException
- NoSuchMethodException
- NoSuchFieldException

2. Unchecked Exception: The classes that inherit the RuntimeException are known as unchecked exceptions. Unchecked exceptions are not checked at compile-time, but they are checked at runtime.

Unchecked Exceptions:

- ArrayIndexOutOfBoundsException
- ClassCastException
- IllegalArgumentException
- IllegalStateException
- NullPointerException
- ArithmeticException

3. Error: Error is irrecoverable. Some example of errors are OutOfMemoryError, VirtualMachineError, AssertionError etc.

Exception Handling Mechanism: In java, exception handling is done using five keywords,

Keyword	Description
try	The "try" keyword is used to specify a block where we should place an exception code. It means we can't use try block alone. The try block must be followed by either catch or finally.
catch	The "catch" block is used to handle the exception. It must be preceded by try block which means we can't use catch block alone. It can be followed by finally block later.
finally	The "finally" block is used to execute the necessary code of the program. It is executed whether an exception is handled or not.
throw	The "throw" keyword is used to throw an exception.
throws	The "throws" keyword is used to declare exceptions. It specifies that there may occur an exception in the method. It doesn't throw an exception. It is always used with method signature.

Exception handling is done by transferring the execution of a program to an appropriate exception handler when exception occurs.

Syntax of Java try-catch:

```
try{
}catch(Exception_class_Name ref){}
public class TryCatchExample
{
    public static void main(String[] args)
    {
        try
        {
            int data=50/0;
            System.out.println("rest of the code");
        }
        catch(ArithmeticException e)
        {
            System.out.println(e);
        }
    }
}
```

Syntax of try-finally block:

```
try{
}finally{}
public class TestFinallyBlock
{
    public static void main(String args[])
    {
```

```

        try
        {
            int data=25/5;
            System.out.println(data);
        }
        catch(NullPointerException e)
        {
            System.out.println(e);
        }
        finally
        {
            System.out.println("finally block is always executed");
        }
        System.out.println("rest of the code...");
    }
}

```

The Java throw keyword is used to throw an exception explicitly.

```

public class TestThrow
{
    public static void validate(int age)
    {
        if(age<18)
        {
            throw new ArithmeticException("Person is not eligible to
vote");
        }
        else
        {
            System.out.println("Person is eligible to vote!!");
        }
    }
    public static void main(String args[])
    {
        TestThrow.validate(13);
        System.out.println("rest of the code...");
    }
}

```

User defined Exception: You can also create your own exception sub class simply by extending java Exception class. You can define a constructor for your Exception sub class (not compulsory) and you can override the toString() function

to display your customized message on catch.

```
class NegativeNumberException extends Exception
{
    public NegativeNumberException (String str)
    {
        super(str);
    }
}
public class TestUser
{
    public static void main(String args[])
    {
        try
        {
            int n = Integer.parseInt(args[0]);
            if (n<0)
            {
                throw new NegativeNumberException (n+" is
negative");
            }
        }
        catch (NegativeNumberException e)
        {
            System.out.println("Caught the exception");
            System.out.println(e.getMessage());
        }
    }
}
```

Java I/O: Java I/O (Input and Output) is used to process the input and produce the output. Java uses the concept of a stream to make I/O operation fast. The java.io package contains all the classes required for input and output operations.

Stream: A stream is a sequence of data. In Java, a stream is composed of bytes. It's called a stream because it is like a stream of water that continues to flow.

In Java, 3 streams are created for us automatically. All these streams are attached with the console.

- 1) System.out: standard output stream
- 2) System.in: standard input stream
- 3) System.err: standard error stream

Java File Class: The File class is an abstract representation of file and directory pathname. A pathname can be either absolute or relative. The File class have

several methods for working with directories and files such as creating new directories or files, deleting and renaming directories or files, listing the contents of a directory etc.

Constructor	Description
File(File parent, String child)	It creates a new File instance from a parent abstract pathname and a child pathname string.
File(String pathname)	It creates a new File instance by converting the given pathname string into an abstract pathname.
File(String parent, String child)	It creates a new File instance from a parent pathname string and a child pathname string.
File(URI uri)	It creates a new File instance by converting the given file: URI into an abstract pathname.

File class methods:

Method	Description
boolean canWrite()	It tests whether the application can modify the file denoted by this abstract pathname.String[]
boolean canExecute()	It tests whether the application can execute the file denoted by this abstract pathname.
boolean canRead()	It tests whether the application can read the file denoted by this abstract pathname.
boolean isAbsolute()	It tests whether this abstract pathname is absolute.
boolean isDirectory()	It tests whether the file denoted by this abstract pathname is a directory.
boolean isFile()	It tests whether the file denoted by this abstract pathname is a normal file.
String getName()	It returns the name of the file or directory denoted by this abstract pathname.
String getParent()	It returns the pathname string of this abstract pathname's parent, or null if this pathname does not name a parent directory.
Path toPath()	It returns a java.nio.file.Path object constructed from the this abstract path.
URI toURI()	It constructs a file: URI that represents this abstract pathname.
File[] listFiles()	It returns an <u>array</u> of abstract pathnames denoting the files in the directory denoted by this abstract pathname
long getFreeSpace()	It returns the number of unallocated bytes in the partition named by this abstract path name.
String[] list(FilenameFilter filter)	It returns an array of strings naming the files and directories in the directory denoted by this abstract pathname that satisfy the specified filter.
boolean mkdir()	It creates the directory named by this abstract pathname.
boolean createNewFile()	It atomically creates a new, empty file named by this abstract pathname if and only if a file with this name does not yet exist.

Java FileInputStream Class: Java FileInputStream class obtains input bytes from a file. It is used for reading byte-oriented data (streams of raw bytes) such as image data, audio, video etc. You can also read character-stream data. But, for reading streams of characters, it is recommended to use FileReader class.

Java FileOutputStream Class: Java FileOutputStream is an output stream used for writing data to a file. If you have to write primitive values into a file, use FileOutputStream class. You can write byte-oriented as well as character-oriented data through FileOutputStream class. But, for character-oriented data, it is preferred to use FileWriter than FileOutputStream.

Java FileReader Class: Java FileReader class is used to read data from the file. It returns data in byte format like FileInputStream class. It is character-oriented class which is used for file handling in java.

Constructor	Description
FileReader(String file)	It gets filename in <u>string</u> . It opens the given file in read mode. If file doesn't exist, it throws FileNotFoundException.
FileReader(File file)	It gets filename in <u>file</u> instance. It opens the given file in read mode. If file doesn't exist, it throws FileNotFoundException.

Methods of FileReader class:

Method	Description
int read()	It is used to return a character in ASCII form. It returns -1 at the end of file.
void close()	It is used to close the FileReader class.

Java FileWriter Class: Java FileWriter class is used to write character-oriented data to a file. It is character-oriented class which is used for file handling in java. Unlike FileOutputStream class, you don't need to convert string into byte array because it provides method to write string directly.

Constructor	Description
FileWriter(String file)	Creates a new file. It gets file name in <u>string</u> .
FileWriter(File file)	Creates a new file. It gets file name in File <u>object</u> .

Methods of FileWriter class:

Method	Description
void write(String text)	It is used to write the string into FileWriter.
void write(char c)	It is used to write the char into FileWriter.
void write(char[] c)	It is used to write char array into FileWriter.
void flush()	It is used to flushes the data of FileWriter.
void close()	It is used to close the FileWriter.

Java DataInputStream Class: Java DataInputStream class allows an application to read primitive data from the input stream in a machine-independent way. Java application generally uses the data output stream to write data that can later be read by a data input stream.

Java DataInputStream class Methods:

Method	Description
int read(byte[] b)	to read the number of bytes from the input stream.
int read(byte[] b, int off, int len)	to read len bytes of data from the input stream.
int readInt()	to read input bytes and return an int value.
byte readByte()	to read and return the one input byte.
char readChar()	to read two input bytes and returns a char value.
double readDouble()	to read eight input bytes and returns a double value.
boolean readBoolean()	to read one input byte and return true if byte is non zero, false if byte is zero.
int skipBytes(int x)	to skip over x bytes of data from the input stream.
String readUTF()	to read a <u>string</u> that has been encoded using the UTF-8 format.
void readFully(byte[] b)	to read bytes from the input stream and store them into the buffer <u>array</u> .
void readFully(byte[] b, int off, int len)	to read len bytes from the input stream.

Java DataOutputStream Class: Java DataOutputStream class allows an application to write primitive Java data types to the output stream in a machine-independent way. Java application generally uses the data output stream to write data that can later be read by a data input stream.

Java DataOutputStream class methods:

Method	Description
int size()	It is used to return the number of bytes written to the data output stream.
void write(int b)	It is used to write the specified byte to the underlying output stream.
void write(byte[] b, int off, int len)	It is used to write len bytes of data to the output stream.
void writeBoolean(boolean v)	It is used to write Boolean to the output stream as a 1-byte value.
void writeChar(int v)	It is used to write char to the output stream as a 2-byte value.
void writeChars(String s)	It is used to write <u>string</u> to the output stream as a sequence of characters.
void writeByte(int v)	It is used to write a byte to the output stream as a 1-byte value.

<code>void writeBytes(String s)</code>	It is used to write string to the output stream as a sequence of bytes.
<code>void writeInt(int v)</code>	It is used to write an int to the output stream
<code>void writeShort(int v)</code>	It is used to write a short to the output stream.
<code>void writeShort(int v)</code>	It is used to write a short to the output stream.
<code>void writeLong(long v)</code>	It is used to write a long to the output stream.
<code>void writeUTF(String str)</code>	It is used to write a string to the output stream using UTF-8 encoding in portable manner.
<code>void flush()</code>	It is used to flushes the data output stream.

SET A

1. Write a java program to accept a number from the user, if number is negative then throw user defined exception —Negative number , otherwise check whether no is prime or not.
2. Write a java program to accept Doctor Name from the user and check whether it is valid or not. (It should not contain digits and special symbol) If it is not valid then throw user defined Exception- Name is Invalid-- otherwise display it.
3. Define a class MyDate (day, month, year) with methods to accept and display a MyDate object. Accept date as dd, mm, yyyy. Throw user defined exception “InvalidDateException” if the date is invalid. Examples of invalid dates: 12 15 2015, 31 6 1990, 29 2 2001.
4. Define class EmailId with members ,username and password. Define default and parameterized constructors. Accept values from the command line Throw user defined exceptions – “InvalidUsernameException” or “InvalidPasswordException” if the username and password are invalid

SET B

1. Write a program to copy the contents of one file to another. Copy the contents in Upper case in second file. Accept two file names using command line program.
2. Write a program to accept a string as command line argument and check whether it is a file or directory. Also perform operations as follows:
 - i. If it is a directory, delete all text files in that directory. Confirm delete operation from user before deleting text files. Also, display a count showing the number of files deleted, if any, from the directory.
 - ii. If it is a file display various details of that file.
3. Write a java program that displays the number of characters, lines and words of a file.
4. Write a java program to accept details of n customers (c_id, cname, address, mobile_no) from user and store it in a file (Use DataOutputStream class).

Display the details of customers by reading it from file.(Use
DataInputStream class)

SET C

1. Write a menu driven program to perform the following operations on a set of integers.
 - i. Load: This operation should generate 10 random integers (2 digit) and display the number on screen.
 - ii. Save: The save operation should save the number to a file “number.txt”.
 - iii. Exit

Signature of instructor: _____ Date: _____

Assignment Evaluation

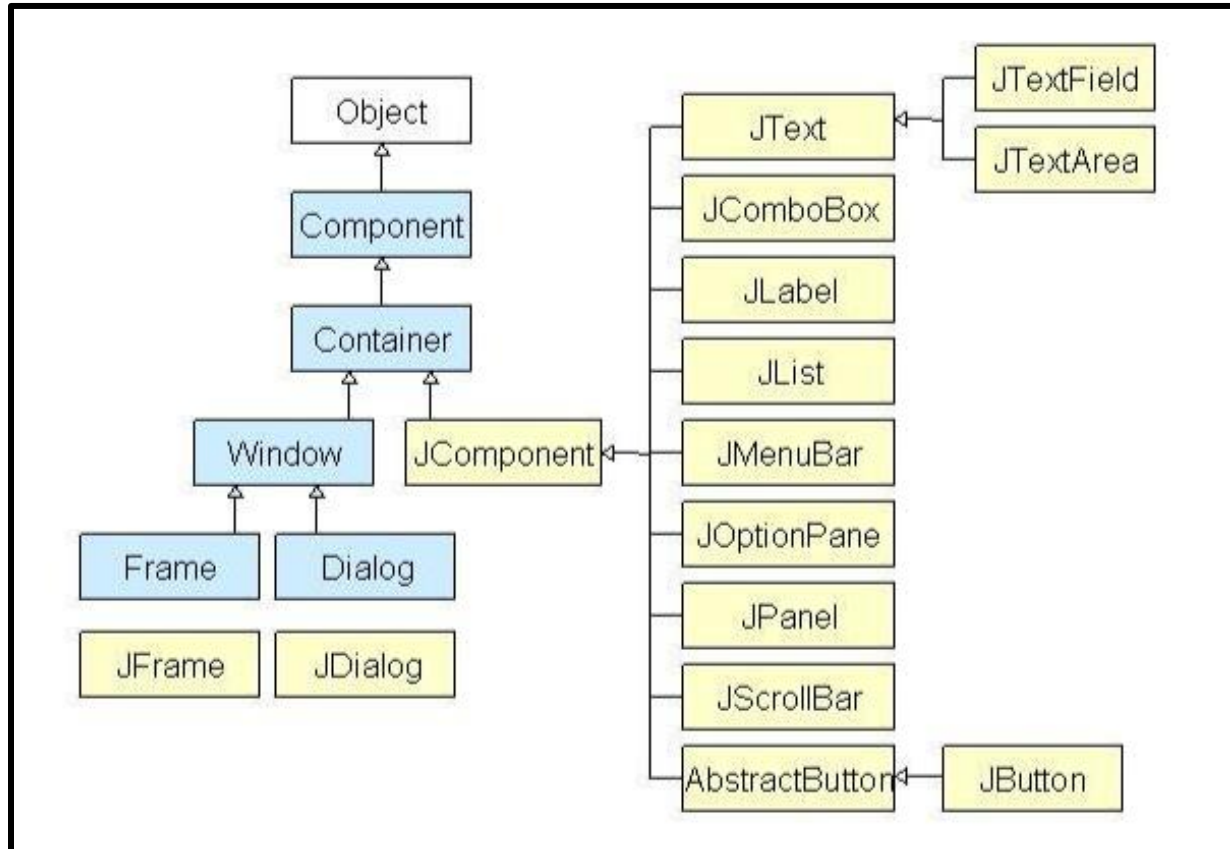
0: Not Done []	1: Incomplete []	2: Late Complete []
3: Needs Improvement []	4: Complete []	5: Well done []

Assignment 5: Swing

Aim: Swing is a part of the Java Foundation Classes (JFC) and provides a rich set of components for building sophisticated and customizable interfaces.

Theory:

Hierarchy of Java Swing classes: The hierarchy of java swing API is given below.



MVC (Model View Controller) Architecture: Swing API architecture follows loosely based MVC architecture in the following manner.

One component architecture is MVC - Model-View-Controller.

The model corresponds to the state information associated with the component.

The view determines how the component is displayed on the screen.

The controller determines how the component responds to the user.

Swing uses a modified version of MVC called "Model-Delegate". In this model the view (look) and controller (feel) are combined into a "delegate".

Because of the Model-Delegate architecture, the look and feel can be changed without affecting how the component is used in a program.

Swing Classes: The following table lists some important Swing classes and their description.

Class	Description
Box	Container that uses a BoxLayout.
JApplet	Base class for Swing applets.
JButton	Selectable component that supports text/image display.
JCheckBox	Selectable component that displays state to user.
JCheckBoxMenuItem	Selectable component for a menu; displays state to user.
JColorChooser	For selecting colors.
JComboBox	For selecting from a drop-down list of choices.
JComponent	Base class for Swing components.
JDesktopPane	Container for internal frames.
JDialog	Base class for pop-up subwindows.
JEditorPane	For editing and display of formatted content.
JFileChooser	For selecting files and directories.
JFormattedTextField	For editing and display of a single line of formatted text.
JFrame	Base class for top-level windows.
JInternalFrame	Base class for top-level internal windows.
JLabel	For displaying text/images.
JLayeredPane	Container that supports overlapping components.
JList	For selecting from a scrollable list of choices.
JMenu	Selectable component for holding menu items; supports text/image display.
JMenuBar	For holding menus.
JMenuItem	Selectable component that supports text/image display.
JOptionPane	For creating pop-up messages.
JPanel	Basic component container.
JPasswordField	For editing and display of a password.
JPopupMenu	For holding menu items and popping up over components.
JProgressBar	For showing the progress of an operation to the user.
JRadioButton	Selectable component that displays state to user; included in ButtonGroup to ensure that only one button is selected.
JRadioButtonMenuItem	Selectable component for menus; displays state to user; included in ButtonGroup to ensure that only one button is selected.
JRootPane	Inner container used by JFrame, JApplet, and others.
JScrollBar	For control of a scrollable area.
JScrollPane	To provide scrolling support to another component.
JSeparator	For placing a separator line on a menu or toolbar.
JSlider	For selection from a numeric range of values.

JSpinner	For selection from a set of values, from a list, a numeric range, or a date range.
JSplitPane	Container allowing the user to select the amount of space for each of two components.
JTabbedPane	Container allowing for multiple other containers to be displayed; each container appears on a tab.
JTable	For display of tabular data.
JTextArea	For editing and display of single-attributed textual content.
TextField	For editing and display of single-attributed textual content on a single line.
JTextPane	For editing and display of multi-attributed textual content.
JToggleButton	Selectable component that supports text/image display; selection triggers component to stay “in”.
JToolBar	Draggable container.
JToolTip	Internally used for displaying tool tips above components.
JTree	For display of hierarchical data.
JViewport	Container for holding a component too big for its display area.
JWindow	Base class for pop-up windows.

Important Containers:

1.JFrame: This is a top-level container which can hold components and containers like panels.

Constructors: JFrame(), JFrame(String title)

Methods

Constructor or Method	Purpose
JFrame()	Creates a frame
JFrame(String title)	Creates a frame with title
setSize(int width, int height)	Specifies size of the frame in pixels
setSize(int width, int height)	Specifies upper left corner
setSize(int width, int height)	Set true to display the frame
setTitle(String title)	Sets the frame title
setDefaultCloseOperation(int mode)	Specifies the operation when frame is closed

Different ways to use this container JFrame:

- i.Create a frame using Swing inside main()
- ii.By creating the object of Frame class (association)
- iii.By extending Frame class (inheritance)

Simple example of Swing by inheritance: We can also inherit the JFrame class, so there is no need to create the instance of JFrame class explicitly.

```
import javax.swing.*;
public class SampleTest extends JFrame
```

```

{
    JFrame f;
    SampleTest()
    {
        JButton b=new JButton("click");//create button
        b.setBounds(130,100,100, 40);
        add(b);//adding button on frame
        setSize(400,500);
        setLayout(null);
        setVisible(true);
    }
    public static void main(String[] args)
    {
        new Simple2();
    }
}

```

JPanel: This is a middle-level container which can hold components and can be added to other containers like frame and panels. The main task of JPanel is to organize components, various layouts can be set in JPanel which provide better organization of components, and however, it does not have a title bar.

Constructors:

JPanel(): creates a new panel with a flow layout

JPanel(LayoutManager l): creates a new JPanel with specified layoutManager

JPanel(boolean isDoubleBuffered): creates a new JPanel with a specified buffering strategy

JPanel(LayoutManager l, boolean isDoubleBuffered): creates a new JPanel with specified layoutManager and a specified buffering strategy

Methods:

add(Component c): Adds a component to a specified container

setLayout(LayoutManager l): sets the layout of the container to the specified layout manager

updateUI(): resets the UI property with a value from the current look and feel.

setUI(PanelsUI ui): sets the look and feel of an object that renders this component.

getUI(): returns the look and feel object that renders this component.

paramString(): returns a string representation of this JPanel.

getUIClassID(): returns the name of the Look and feel class that renders this component.

getAccessibleContext(): gets the AccessibleContext associated with this JPanel.

import java.awt.*;

import java.awt.event.*;

```

import javax.swing.*;
public class PanelTest extends JFrame
{
    private JButton b, b1, b2;
    private JLabel l;
    public PanelTest()
    {
        l = new JLabel("panel label");
        b = new JButton("button1");
        b1 = new JButton("button2");
        b2 = new JButton("button3");
        JPanel p = new JPanel();
        p.add(b);
        p.add(b1);
        p.add(b2);
        p.add(l);
        p.setBackground(Color.red);
        add(p);
        setSize(300, 300);
        show();
    }
    public static void main(String[] args)
    {
        PanelTest p = new PanelTest();
    }
}

```

Layout Manager: The job of a layout manager is to arrange components on a container. Each container has a layout manager associated with it. To change the layout manager for a container, use the `setLayout()` method.

Syntax: `setLayout(LayoutManager obj)`

The predefined managers are listed below:

1. FlowLayout
- 2.BorderLayout
- 3.GridLayout
4. BoxLayout
- 5.CardLayout
- 6.GridBagLayout

Examples:

```

JPanel p1 = new JPanel();
p1.setLayout(new FlowLayout());

```

```
p1.setLayout(new BorderLayout());  
p1.setLayout(new GridLayout(3,4));
```

JLabel: With the JLabel class, you can display unselectable text and images.

Constructors: JLabel(Icon i), JLabel(Icon I , int n), JLabel(String s), JLabel(String s, Icon i, int n), JLabel(String s, int n), JLabel()

The int argument specifies the horizontal alignment of the label's contents within its drawing area; defined in the SwingConstants interface (which JLabel implements): LEFT (default), CENTER, RIGHT, LEADING, or TRAILING.

Methods: void setText(String), String getText(), void setIcon (Icon), Icon getIcon(), void setDisabledIcon(Icon), Icon getDisabledIcon(), void setHorizontalAlignment(int), void setVerticalAlignment(int), int getHorizontalAlignment(), int getVerticalAlignment()

JButton: A Swing button can display both text and an image. The underlined letter in each button's text shows the mnemonic which is the keyboard alternative.

Constructors: JButton(Icon I), JButton(String s), JButton(String s, Icon I).

Methods: void setDisabledIcon(Icon), void setPressedIcon(Icon), void setSelectedIcon(Icon), void setRolloverIcon(Icon), String getText()

JCheckBox:

Constructors: JCheckBox(Icon i), JCheckBox(Icon i, boolean state),

JCheckBox(String s), JCheckBox(String s, boolean state), JCheckBox(String s, Icon i), JCheckBox(String s, Icon I, boolean state)

Methods: void setSelected(boolean state) String getText(), void setText(String s)

JRadioButton:

Constructors: JRadioButton (String s), JRadioButton(String s, boolean state),

JRadioButton(Icon i), JRadioButton(Icon i, boolean state), JRadioButton(String s, Icon i), JRadioButton(String s, Icon i, Boolean state), JRadioButton()

To create a button group- ButtonGroup(), Adds a button to the group, or removes a button from the group, void add(AbstractButton), void remove(AbstractButton)

JComboBox:

Constructors: JComboBox()

Methods: void addItem(Object), Object getItemAt(int), Object getSelectedItem(), int getItemCount()

JList:

Constructor: JList(ListModel)

Methods: boolean isSelectedIndex(int), void setSelectedIndex(int), void setSelectedIndices(int[]), void setSelectedValue(Object, boolean), void setSelectedInterval(int, int), int getSelectedIndex(), int getMinSelectionIndex(), int getMaxSelectionIndex(), int[] getSelectedIndices(), Object getSelectedValue(), Object[] getSelectedValues()

Text classes: All text related classes are inherited from JTextComponent class

- a. JTextField: Creates a text field. The int argument specifies the desired width in columns. The String argument contains the field's initial text. The Document argument provides a custom document for the field.

Constructors: JTextField(), JTextField(String), JTextField(String, int)
JTextField(int), JTextField(Document, String, int)

- b. JPasswordField:

Constructors: JPasswordField(), JPasswordField(String),
JPasswordField(String, int) , JPasswordField(int), JPasswordField(Document,
String, int)

Methods:

1. Set or get the text displayed by the text field: void setText(String) String
getText()
 2. Set or get the text displayed by the text field: char[] getPassword()
 3. Set or get whether the user can edit the text in the text field: void
setEditable(boolean) boolean isEditable()
 4. Set or get the number of columns displayed by the text field. This is really
just a hint for computing the field's preferred width: void setColumns(int);
int getColumns()
 6. Get the width of the text field's columns. This value is established implicitly
by the font: int getColumnWidth()
 7. Set or get the echo character i.e. the character displayed instead of the
actual characters typed by the user: void setEchoChar(char) char
getEchoChar()
- c. JTextArea: Represents a text area which can hold multiple lines of text
Constructors: JTextArea (int row, int cols), JTextArea (String s, int row, int
cols)
Methods: void setColumns (int cols), void setRows (int rows), void
append(String s), void setLineWrap (boolean)

Event Handling on components:

Java has two types of events:

- 1.Low-Level Events: Low-level events represent direct communication from user. A low level event is a key press or a key release, a mouse click, drag, move or

release, and so on.

Following are low level events:

Event	Description
ComponentEvent	Indicates that a component object (e.g. Button, List, TextField) is moved, resized, rendered invisible or made visible again.
FocusEvent	Indicates that a component has gained or lost the input focus.
KeyEvent	Generated by a component object (such as TextField) when a key is pressed, released or typed.
MouseEvent	Indicates that a mouse action occurred in a component. E.g. mouse is pressed, releases, clicked (pressed and released), moved or dragged.
ContainerEvent	Indicates that a container's contents are changed because a component was added or removed.
WindowEvent	Indicates that a window has changed its status. This low level event is generated by a Window object when it is opened, closed, activated, deactivated, iconified, deiconified or when focus is transferred into or out of the Window.

High-Level Events: High-level (also called as semantic events) events encapsulate the meaning of a user interface component. These include following events:

Event	Description
ActionEvent	Indicates that a component-defined action occurred. This high- level event is generated by a component (such as Button) when the component-specific action occurs (such as being pressed).
AdjustmentEvent	The adjustment event is emitted by Adjustable objects like scrollbars.
ItemEvent	Indicates that an item was selected or deselected. This high- level event is generated by an ItemSelectable object (such as a List) when an item is selected or deselected by the user.
TextEvent	Indicates that an object's text changed. This high-level event is generated by an object (such as TextComponent) when its text changes.

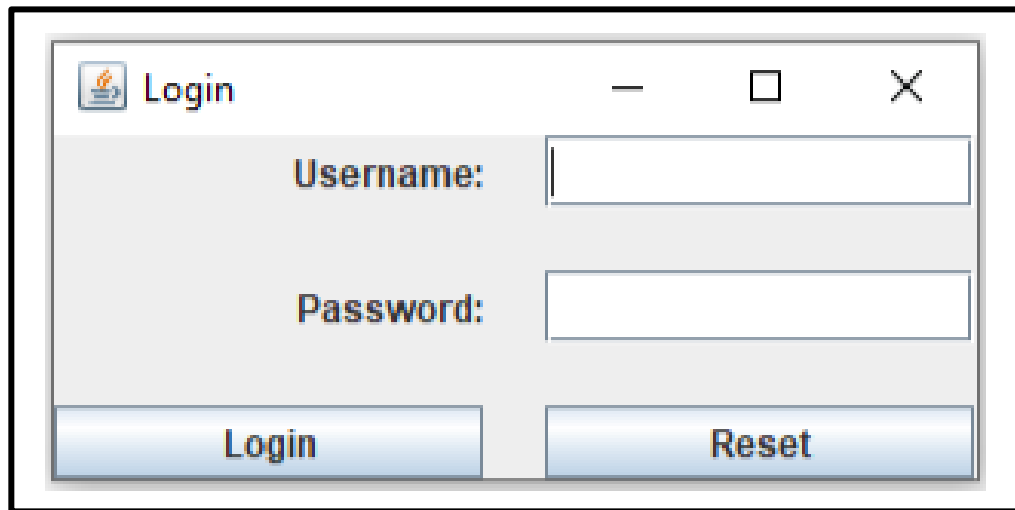
The following table lists the events, their corresponding listeners and the method to add the listener to the component.

Event	Event Source	Event Listener	Method to add listener to event source
Low – Level Events			
ComponentEvent	Component	ComponentListener	addComponentListener()
FocusEvent	Component	FocusListener	addFocusListener()
KeyEvent	Component	KeyListener	addKeyListener()

MouseEvent	Component	MouseListener MouseMotionListener	addMouseListener() addMouseMotionListener()
ContainerEvent	Container	ContainerListener	addContainerListener()
WindowEvent	Window	WindowListener	addWindowListener()
High-level Events			
ActionEvent	Button List MenuItem TextField	ActionListener	addActionListener()
ItemEvent	Choice CheckBox CheckBoxM enuItemList	ItemListener	addItemListener()
AdjustmentEvent	Scrollbar	AdjustmentListener	addAdjustmentListener()
TextEvent	TextField TextArea	TextListener	addTextListener()

SET A

1. Write a java program to design a following GUI. Use appropriate Layout and Components.



2. Write a java program to design a following GUI. Use appropriate Layout and Components.

Vaccination Details

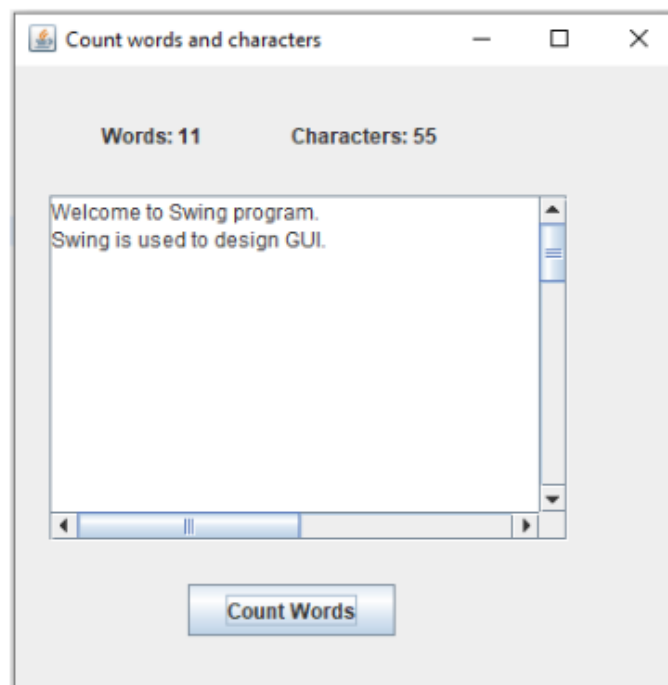
Name:

Dose	Vaccine
☐ 1 st Dose	☐ Covishield
☐ 2 nd Dose	☐ Covaxin
	☐ Sputnik V

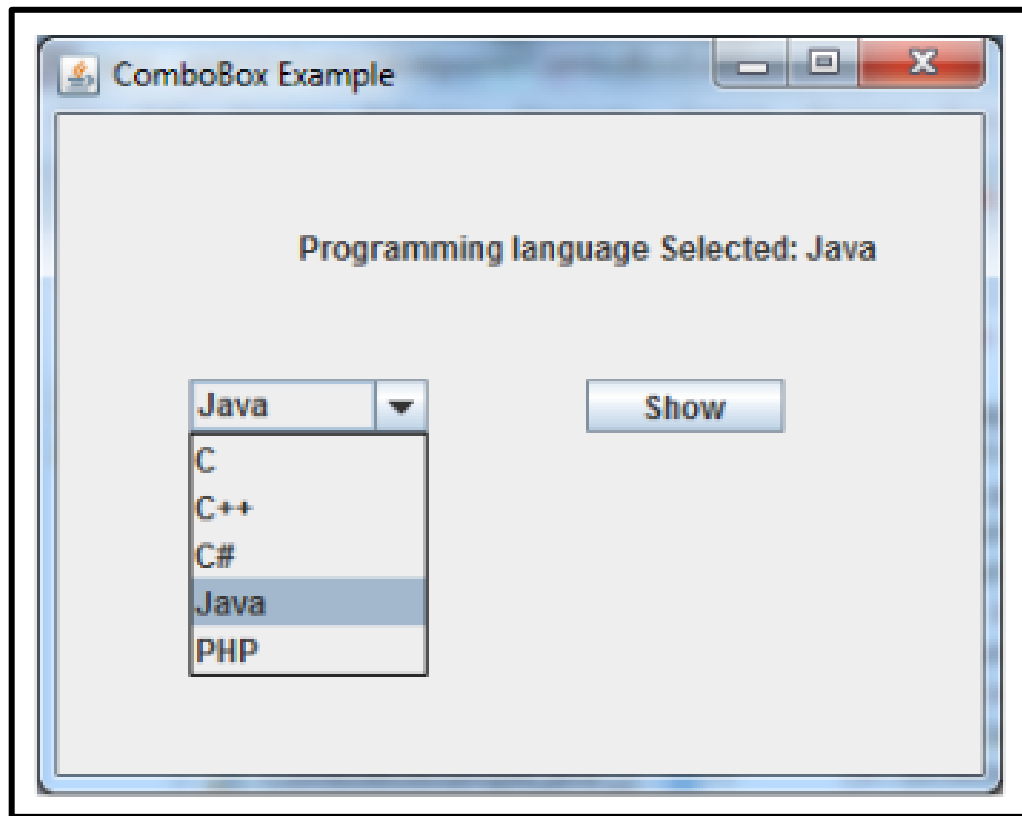
Name : _____ 1st Dose: _____ 2nd Dose: _____

Vaccine: _____

3. Write a program to design following GUI using JTextArea. Write a code to display number of words and characters of text in JLabel. Use JScrollPane to get scrollbars for JTextArea.



4. Write a Program to design following GUI by using swing component JComboBox. On click of show button display the selected language on JLabel.



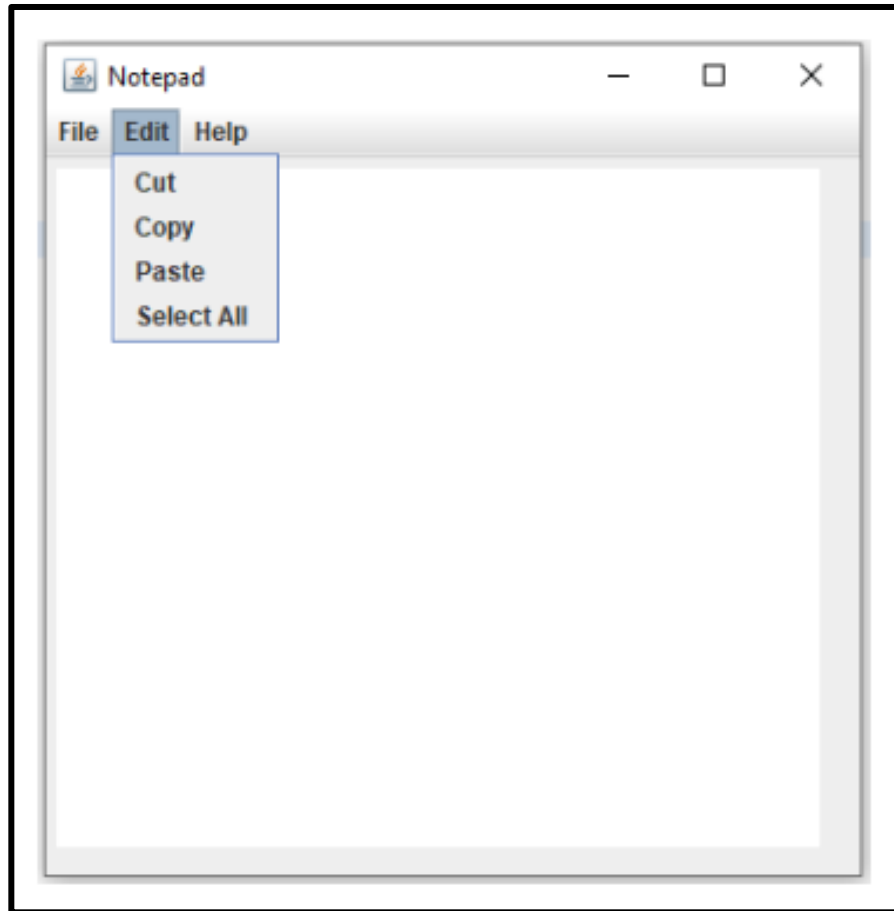
SET B

1. Implement event handling for SET A 1. Verify username and password in 3 attempts. Display dialog box "Login successful" on success or display "Username or Password is in correct". After 3 attempts display "Login Failed". On reset button clear the fields of text box.
2. Implement event handling for SET A 2. Display selected Name, Vaccine. If 1st Dose is taken then write Yes otherwise write No.
3. Write a java program to create the following GUI using Swing components.



SET C

1. Write a program to design and implement following GUI.



Signature of instructor: _____ Date: _____

Assignment Evaluation

0: Not Done []

1: Incomplete []

2: Late Complete []

3: Needs Improvement []

4: Complete []

5: Well done []